技術報告 ／ Technical Report

Logo Data Engine
多品牌批次資料管道
設計與執行說明

```
./logo_data_engine/run_multibrand_batch.sh \
  --run-name      batch_10_vlm3_sam3          \
  --target-records 10                          \
  --use-vlm                                    \
  --vlm-model-id  llava-hf/llava-1.5-7b-hf    \
  --with-sam3                                  \
  --cuda-devices  1,2
```

執行批次：**batch_10_vlm3_sam3**

執行日期：2026-04-04

工作目錄：`/raid/ming/logo/logo_data_engine`

執行環境：Conda `logo_sam3` ／ CUDA Devices 1, 2

目錄

1 執行概覽 3

1.1 批次參數一覽 3

1.2 AI 模型堆疊（Preflight 結果） 3

1.3 整體數字摘要 4

2 Pipeline 全流程架構 5

2.1 兩階段 DB 設計（Staging → Engine） 5

2.2 流程圖總覽 5

3 各步驟詳細說明 6

Step 0 Preflight — 模型可用性預檢 6

Step 1 Batch Plan — 計算 Limit per Pair 7

Step 2 Collector — 多品牌圖片爬取 7

Step 3 Balance（第一次）— Oversample 候選池 8

Step 4 Ontology — 品牌知識庫抓取 8

Step 5–6 Staging Seed & Ingest 9

Step 7 Quality Gate — 五道品質過濾 9

Step 8–9 Export Passed & 最終平衡 12

Step 10 SAM3 Logo Segmentation 12

Step 11 Phase1 Workflow — 核心標注引擎 14

Step 12 Export Knowledge Base 17

Step 13 Analyze Run — 批次分析報告 18

4 輸出目錄結構 19

5 Knowledge Base 欄位完整說明 20

6 本批次統計結果 22

7 Review Queue 與下一步行動 23

1 執行概覽

1.1 批次參數一覽

參數	指令列選項	值	說明
批次名稱	`--run-name`	`batch_10_vlm3_sam3`	輸出目錄名稱
目標筆數	`--target-records`	`10`	最終品質過關後保留的記錄數
Oversample 倍數	（預設）	`3`	候選池大小 = 10 × 3 = 30 筆
VLM 啟用	`--use-vlm`	`true`	啟用 LLaVA 圖像描述後端
VLM 模型	`--vlm-model-id`	`llava-hf/llava-1.5-7b-hf`	LLaVA 1.5 7B 模型
SAM3 分割	`--with-sam3`	`true`	啟用 SAM3 精細 Logo Mask 分割
CUDA 設備	`--cuda-devices`	`1,2`	設定 CUDA_VISIBLE_DEVICES=1,2
Object-First	（預設開啟）	`true`	先找物件框再找 Logo
OCR	（預設開啟）	`true`	PaddleOCR 讀取 Logo 文字
Resume	（預設開啟）	`true`	已存在記錄自動跳過
Conda 環境	`--env-name`	`logo_sam3`	執行所用的 Conda 虛擬環境

1.2 AI 模型堆疊（Preflight 實際檢測結果）

每次批次執行的第一步是 **Preflight 預檢**，系統會逐一初始化所有 AI 模型並回報可用性。以下為本批次實際 preflight 結果（`meta/preflight.json`）：

模組	模型 ID	狀態	用途
GroundingDINO	`IDEA-Research/grounding-dino-tiny`	√ 可用	物件偵測、Logo BBox 定位
PaddleOCR	`en` （英文模型）	√ 可用	Logo 區域文字辨識
CLIP（品質評分）	`openai/clip-vit-base-patch32`	√ 可用	Quality Gate：判斷圖片是否含 Logo
CLIP（品牌比對）	`openai/clip-vit-base-patch32`	√ 可用	Logo Crop 對品牌名稱庫做 Retrieval
BLIP（圖說）	`Salesforce/blip-image-captioning-base`	√ 可用	整圖與 Logo Crop 的自然語言描述（備用）
LLaVA VLM	`llava-hf/llava-1.5-7b-hf`	√ 可用	主要 Caption/QA 後端（`--use-vlm`）
YOLO-World	`yolov8s-worldv2.pt`	√ 可用	Quality Gate：Logo 預篩（Prescreener）
SAM3	`Sam3LogoSegmenter`	√ 可用（CUDA）	Logo Mask 精細像素級分割

注意：YOLO-World 信心閾值

本批次 YOLO-World prescreener 的偵測閾值設定為 **0.3**（即 `conf_threshold=0.3`）。若圖片中 YOLO-World 未找到任何高於閾值的 Logo，不會直接過濾，而是標記為 `soft_pass` 並在記錄上附加 `difficulty_flags: ["no_logo_prescreen_detection", "hard_logo_candidate"]`。

1.3 整體數字摘要

指標	數值	說明
設定的品牌/類別 Pairs 數	17	batch_plan 計算出的 pair 總數
每 Pair 抓取上限	2	由 batch_plan 推算：30 ÷ 17 ≈ 2
Collector 實際抓取	34	multi_brand_fetcher.py 從 Poshmark 爬回
Oversample 候選池	30	balance-records 從 34 筆均勻選出
Staging Gate 輸入	40	含歷史批次圖片（staging DB 累積）
Quality Gate 通過	10	30 筆因 near_duplicate 過濾
最終平衡後保留	10	= target-records，目標達成
SAM3 分割成功	10 / 10	10 筆全數成功（ok），0 筆失敗
正式 DB 品牌筆數	13	seed-ontology 寫入 engine DB
Logo Instances 產生	10	phase1 annotation 插入
Knowledge Base 記錄數	20	JOIN image_records × logo_instances
Review Queue 筆數	20	全數標記 needs_review（confidence < 0.9）
品牌種類	6	Adidas, Disney, Under Armour, Lululemon, New Balance, Starbucks
類別種類	3	apparel, shoes, mugs
資料來源	1	poshmark（marketplace）
目標達成	**√ True**	**target_achieved = true**

2 Pipeline 全流程架構

2.1 兩階段 DB 設計（Staging → Engine）

本 Pipeline 使用 **兩個獨立的 SQLite 資料庫**，以確保品質過濾前後的資料完全隔離：

- **Staging DB**（`staging/logo_engine.db`）： 用於 Step 5–9 的 oversample 候選管理與品質篩選。所有候選圖片先進入此 DB， 經 Quality Gate 後，只有 `quality_status=passed` 的記錄才進入下一階段。
- **Engine DB**（`engine/logo_engine.db`）： 最終 10 筆圖片的正式資料庫。Step 10（SAM3）以後的所有標注、品牌資料、 Logo Instances 均寫入此 DB，並作為 Knowledge Base 的後端儲存。

2.2 流程圖總覽

```
指令列參數解析
      │
      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  Step 0：preflight（→ staging DB）                                        │
│  初始化所有 AI 模型，回報可用性                                            │
└──────────────────────────────────────────────────────────────────────────┘
      │  meta/preflight.json
      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  Step 1：batch-plan                                                       │
│  計算 17 pairs × limit_per_pair = 2，目標候選池 30 筆                      │
└──────────────────────────────────────────────────────────────────────────┘
      │  meta/batch_plan.json
      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  Step 2：collector-fetch-products --all                                   │
│  multi_brand_fetcher.py → Poshmark 爬取 → 34 筆原始記錄 + 60+ 張圖片      │
└──────────────────────────────────────────────────────────────────────────┘
      │  meta/raw_records.json + fetch/images/*.jpeg
      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  Step 3：balance-records（第一次）target=30                                │
│  從 34 筆均勻抽樣 30 筆，確保品牌/類別平衡                                 │
└──────────────────────────────────────────────────────────────────────────┘
      │  meta/candidate_records.json
      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  Step 4：ontology-fetch-brands                                            │
│  brand_data_fetcher.py → Wikidata → 品牌知識庫（aliases, industry...）     │
└──────────────────────────────────────────────────────────────────────────┘
      │  brand/brand_records.json
      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  Step 5：seed-ontology（staging DB）                                      │
│  品牌資料寫入 staging DB                                                   │
└──────────────────────────────────────────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  Step 6：ingest-images（staging DB）                                      │
│  30 筆候選圖片 + pHash 去重後寫入 staging DB                               │
└──────────────────────────────────────────────────────────────────────────┘
      │  40 筆進入 Quality Gate（含歷史）
      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  Step 7：gate（staging DB）— 5 道品質關卡                                  │
│  ① size_format：40通過 ② blur：40通過 ③ dedupe：10通過/30失敗(near_dup)    │
│  ④ clip_logo：6通過/4軟通過 ⑤ logo_prescreen：10軟通過                     │
│  → 最終：10 passed / 30 filtered                                          │
└──────────────────────────────────────────────────────────────────────────┘
      │  meta/staging_gate.json
      ▼
┌──────────────────────────────────────────────────────────────────────────┐
```

```
│  Step 8：export-image-records（quality_status=passed）                        │
│  從 staging DB 撈出 10 筆過關記錄                                              │
└──────────────────────────────────────────────────────────────────────────────┘
     │  meta/passed_records.json
     ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│  Step 9：balance-records（第二次）target=10                                    │
│  確保最終 10 筆品牌分佈均勻                                                    │
└──────────────────────────────────────────────────────────────────────────────┘
     │  fetch/records.json（最終 10 筆）
     │
     ├──────────────────────────────────────────────────────────────┐
     ▼                                                              ▼
┌──────────────────────────────────────────┐      品牌資料繼續流向 Step 11
│  Step 10：segment-sam3（engine DB）        │
│  對每張圖執行 3 層 Grounding + SAM3 分割：  │
│  L1: GroundingDINO 物件框（object-first）  │
│  L2: GroundingDINO Logo BBox              │
│  L3: SAM3 精細像素級 Mask                  │
│  → 10/10 成功                              │
└──────────────────────────────────────────┘
     │  segment/records_with_logo_masks.json
     │  segment/masks/*.png
     │  segment/visualizations/*.png
     ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│  Step 11：phase1-workflow（engine DB）                                         │
│  11a. seed-ontology → 13 品牌寫入 engine DB                                    │
│  11b. ingest-images → 10 筆圖片寫入 engine DB                                  │
│  11c. gate（engine DB） → 10/10 通過                                           │
│  11d. annotate（SAM3 Enrich 模式）：                                           │
│       ├─ PaddleOCR → Logo BBox 文字辨識                                        │
│       ├─ CLIP Retrieval → Logo Crop vs 品牌名稱庫                              │
│       ├─ BLIP Captioning → 整圖 + Logo Crop 描述                               │
│       ├─ LLaVA VLM → 整圖 + Logo Crop 自然語言描述（主要 Caption）             │
│       └─ merge_signals() → 融合所有信號決定品牌歸屬                            │
│  11e. review-queue → 輸出低信心記錄清單（全 20 筆 needs_review）               │
└──────────────────────────────────────────────────────────────────────────────┘
     │
     ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│  Step 12：export-kb                                                            │
│  JOIN image_records × logo_instances → 20 筆 knowledge_base.json               │
└──────────────────────────────────────────────────────────────────────────────┘
     │
     ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│  Step 13：analyze-run + summary                                                │
│  產生 analysis.md、summary.json、meta/summary.json                             │
└──────────────────────────────────────────────────────────────────────────────┘
```

3 各步驟詳細說明

0 Preflight — 模型可用性預檢

目的：在任何資料處理開始前，逐一初始化所有 AI 模型並確認其可用性。若任何核心模型初始化失敗，後續步驟將無法正確執行。

對應指令：

```
python -m logo_data_engine \
    --db staging/logo_engine.db \
    preflight \
    --with-sam3 \
    --use-vlm --vlm-model-id llava-hf/llava-1.5-7b-hf
```

初始化順序與行為：

1. **GroundingDINO**：載入 `IDEA-Research/grounding-dino-tiny`，測試能否前向傳播
2. **PaddleOCR**：初始化英文 OCR 模型（`lang="en"`）
3. **CLIP Logo Scorer**：載入 `openai/clip-vit-base-patch32`，用於 Quality Gate
4. **CLIP Brand Retriever**：同上模型，用於品牌 Retrieval
5. **BLIP Captioner**：載入 `Salesforce/blip-image-captioning-base`
6. **LLaVA VLM**：載入 `llava-hf/llava-1.5-7b-hf`（7B 參數，需 GPU 記憶體）
7. **YOLO-World Prescreener**：載入 `yolov8s-worldv2.pt`，閾值 0.3
8. **SAM3**：初始化 `Sam3LogoSegmenter`（`device=cuda`，`object_first=True`）

實際輸出（`meta/preflight.json`）：

```
{"with_sam3": true, "use_vlm": true,
 "grounding_dino":    {"enabled": true, "available": true, "model_id": "IDEA-Research/grounding-d
 "paddleocr":         {"enabled": true, "available": true},
 "clip_logo_gate":    {"enabled": true, "available": true},
 "clip_retrieval":    {"enabled": true, "available": true},
 "blip_caption":      {"enabled": true, "available": true},
 "vlm":               {"enabled": true, "available": true, "model_id": "llava-hf/llava-1.5-7b-hf"
 "yolo_prescreen":    {"enabled": true, "available": true, "threshold": 0.3},
 "sam3":              {"enabled": true, "available": true, "device": "cuda", "object_first": true
```

1 Batch Plan — 計算 Limit per Pair

目的：根據 `--target-records × oversample_factor = 10 × 3 = 30`，計算每個品牌/類別 Pair 應抓幾筆（`recommended_limit_per_pair`），使總候選數接近 30。

對應指令：

```
python -m logo_data_engine --db staging/logo_engine.db \
    batch-plan --target 30
```

計算邏輯：系統讀取所有設定的 brand/category pairs（共 17 個），計算 `⌈30 ÷ 17⌉ = 2`。

實際輸出（`meta/batch_plan.json`）：

```
{"target": 30, "configured_pairs": 17, "recommended_limit_per_pair": 2}
```

2 Collector — 多品牌圖片爬取

目的：呼叫 `multi_brand_fetcher.py`，以 `--all` 模式遍歷所有設定的品牌/類別 Pairs，從 Poshmark 等 Marketplace 爬取商品圖片及 metadata。

對應指令：

```
python multi_brand_fetcher.py \
    --all \
    --limit 2 \
    --output meta/raw_records.json \
    --image-dir fetch/images/
```

Poshmark 爬取機制：對每個 `(brand, category)` pair，構造搜尋 URL（例如 `https://poshmark.com/search?query=adidas+shirt&type=listings`），爬取前 N 筆商品，下載縮圖並記錄 metadata。

每筆原始記錄的欄位（raw_records.json）：

欄位	範例值	說明
`brand`	`"Adidas"`	品牌標籤
`category`	`"apparel"`	類別標籤
`source`	`"poshmark"`	資料來源名稱
`source_channel`	`"marketplace"`	管道類型
`source_kind`	`"marketplace"`	來源種類（影響 soft-pass 判斷）
`product_name`	`"Adidas Shirt"`	商品名稱
`product_url`	`"https://poshmark.com/listing/..."`	商品頁面 URL
`image_url`	`"https://di2ponv0v5otw.cloudfront.net/..."`	圖片 CDN URL
`local_image_path`	`"fetch/images/adidas_69c47adb.jpg"`	本地儲存路徑
`global_product_id`	`"69c47adb006e4345ae58f0fb"`	Poshmark 商品 ID
`crawl_timestamp_utc`	`"2026-04-04T01:13:44.996800+00:00"`	爬取時間戳
`image_download_status`	`"downloaded"`	下載狀態

實際輸出（`meta/collector.json`）：

```
{"output": "meta/raw_records.json", "records": 34, "pairs": 17}
```

圖片命名規則

下載的圖片以 `<brand>_<global_product_id>.<ext>` 命名，存放於 `fetch/images/`。例如：`adidas_69c47adb006e4345ae58f0fb.jpg`、`disney_69c9e3a9d6c0dcd7953eee8e.jpeg`。

3 Balance（第一次）— Oversample 候選池

目的：從 Collector 抓回的 34 筆中，均勻抽取 30 筆，使品牌與類別的分佈盡量平衡，避免某品牌獨佔候選池。

對應指令：

```
python -m logo_data_engine --db staging/logo_engine.db \
    balance-records \
    --input  meta/raw_records.json \
    --output meta/candidate_records.json \
    --target 30
```

平衡策略：以 `(brand, category, source)` 為分層鍵，在各層中均勻取樣，優先補足數量不足的稀有組合。

實際輸出（`meta/balance.json`）：

```
{"input": "meta/raw_records.json", "output": "meta/candidate_records.json",
 "seen": 34, "selected": 30, "target": 30}
```

4 Ontology — 品牌知識庫抓取

目的：呼叫 `brand_data_fetcher.py`，對每個出現在候選記錄中的品牌名稱，查詢 Wikidata 取得結構化知識（別名、行業、國家、母公司等），作為後續品牌比對與 Prompt 建構的依據。

對應指令：

```
python brand_data_fetcher.py \
    --product-records meta/candidate_records.json \
    --output          brand/brand_records.json
```

Wikidata 查詢流程：

1. 對每個品牌名稱執行 Wikidata 搜尋（exact match 優先）
2. 從匹配的 entity 中提取：`aliases_en`、`industries`、`countries`、`parent_organizations`、`inception_year`、`official_websites`
3. 將結果序列化為 `brand_records.json`

實際品牌記錄範例（Adidas）：

```
{"query": "Adidas", "matched": true, "matched_by": "exact_or_top_wikidata_search",
 "entity_id": "Q12358242", "label": "Adidas",
 "description": "trade mark of Adidas AG",
 "aliases_en": ["Adidas brand", "Cloudfoam", "Adidas Cloudfoam",
                "Cloudfoam Comfort", "Cloudfoam Comfort shoes"],
 "wikidata_url": "https://www.wikidata.org/wiki/Q12358242",
 "official_websites": ["https://www.adidas-group.com/"],
 "parent_organizations": [],
 "countries": ["Germany"],
 "industries": ["sportswear"],
 "inception_year": "1924"}
```

這些 aliases_en 在 Step 10（SAM3）的 Prompt 建構中扮演關鍵角色：GroundingDINO 會以 `"Adidas logo"`、`"Cloudfoam logo"`、`"Adidas Cloudfoam logo"` 等多種變體同時偵測， 提高 Logo 定位的召回率。

實際輸出（`meta/ontology.json`）：

```
{"output": "brand/brand_records.json", "dry_run": false}
```

5 6 Staging Seed & Ingest — 寫入暫存資料庫

Step 5（seed-ontology）：將 `brand_records.json` 中 `matched=true` 的品牌寫入 staging DB 的 `brand_records` 資料表，tier 設定為 `"silver"`。

Step 6（ingest-images）：對 `candidate_records.json` 中每筆記錄：

1. 計算圖片的 pHash（感知雜湊，用於後續去重）
2. 若 pHash 在既有記錄中已存在（Hamming distance ≤ 6），跳過
3. 否則寫入 staging DB `image_records` 資料表，初始 `quality_status=NULL`

實際輸出（`meta/staging_ingest.json`）：

```
{"seen": 30, "inserted": 30, "skipped_existing": 0,
 "skipped_duplicate": 0, "phash_failures": 0}
```

7 Quality Gate — 五道品質過濾

目的：對 staging DB 中所有 `quality_status IS NULL` 的圖片依序執行 5 道品質關卡， 更新 `quality_status`（`passed` 或 `filtered`）並記錄詳細的 gate 結果。

對應指令：

```
python -m logo_data_engine --db staging/logo_engine.db \
    gate --all --report
```

本批次實際 Gate 結果總覽：

關卡名稱	passed	soft_pass	failed	skipped	失敗原因
① size_format	40	0	0	0	—
② blur	40	0	0	0	—
③ dedupe（pHash）	10	0	30	0	`near_duplicate`
④ clip_logo	6	4	0	0	soft_pass（低分但來自 marketplace）
⑤ logo_prescreen	0	10	0	0	soft_pass（YOLO 未偵測到 Logo）
最終結果	**10 passed**	**30 filtered**			

① 尺寸與格式關卡（size_format）

判斷條件

- 最小寬度：**256px**（`min_width=256`）
- 最小高度：**256px**（`min_height=256`）
- 允許格式：**JPEG、PNG、WEBP**
- 失敗代碼：`too_small` 或 `unsupported_format`

本批次 40 張圖片全部通過，代表 Poshmark 圖片均符合最低解析度要求。Gate 輸出範例：

```
{"gate": "size_format", "outcome": "passed", "width": 300, "height": 300, "format": "JPEG"}
```

② **模糊偵測關卡（blur）**

> **計算方法**
>
> 採用 **Laplacian Variance** 衡量圖像銳利度。 計算方式：將圖片轉為灰階後，對每個像素計算其 4-鄰域 Laplacian（ `上 + 下 + 左 + 右 − 4×中心` ），取整體變異數。 閾值 **45.0**：低於此值視為過度模糊（ `too_blurry` ）。

本批次實際 blur_score 範例（Disney mug 圖）：

```
{"gate": "blur", "outcome": "passed", "blur_score": 1383.24365234375}
```

此分數遠高於閾值 45.0，說明圖片相當清晰。通過後，blur_score 對 quality_score 的貢獻為 `min(1.0, blur_score / 90.0)`。

③ **感知哈希去重關卡（dedupe）**

> **本批次主要過濾關卡**
>
> 本批次共有 **30 筆**因此關卡被過濾，是最大的篩選瓶頸。

計算圖片的 pHash（64-bit 感知哈希），與所有已通過的圖片做 Hamming Distance 比較。 若 Hamming Distance ≤ **6**，視為近似重複（ `near_duplicate` ）。

Hamming Distance 計算方式（XOR 後計算 bit 數）：

```
distance = bin(int(phash_a, 16) ^ int(phash_b, 16)).count('1')
```

Gate 輸出範例（通過）：

```
{"gate": "dedupe", "outcome": "passed", "phash_distance": null, "threshold": 6}
```

Gate 輸出範例（失敗）：

```
{"gate": "dedupe", "outcome": "failed", "phash_distance": 2, "threshold": 6}
```

為何有 30 筆重複？

staging DB 為累積式設計，含有先前批次已下載的圖片。 本次 30 筆重複來自 Poshmark 上相同商品圖在不同批次中重複出現， 系統正確地以 pHash 偵測並過濾，確保最終資料集不含重複圖。

④ CLIP Logo 品質關卡（clip_logo）

使用 `openai/clip-vit-base-patch32` 計算整張圖片與 prompt `"a photo with a brand logo"` 的餘弦相似度分數。

決策邏輯

- 分數 ≥ **0.22**（`clip_threshold`）：**passed**
- 分數 < 0.22 且 ≥ **0.08**（`clip_soft_floor`）且為 marketplace/ecommerce 可信來源： **soft_pass**，標記 `difficulty_flags: ["low_clip_logo_score"]`
- 分數 < 0.08 或非可信來源：**filtered**（`low_clip_logo_score`）

Gate 輸出範例（soft_pass）：

```
{"gate": "clip_logo", "outcome": "soft_pass",
 "score": 0.21255092322826385, "threshold": 0.22,
 "soft_floor": 0.08, "model_id": "openai/clip-vit-base-patch32"}
```

本批次 10 筆通過記錄中：6 筆 **passed**、4 筆 **soft_pass**（分數介於 0.08–0.22 之間，均來自 Poshmark marketplace）。

⑤ YOLO-World Logo 預篩關卡（logo_prescreen）

使用 `yolov8s-worldv2.pt` 對整張圖做 open-vocabulary 目標偵測，判斷是否存在 Logo 相關物件（置信度 ≥ 0.3）。

決策邏輯

- 有偵測到 Logo（置信度 ≥ 0.3）：**passed**，另標記：
  - BBox 面積 < 2% 圖面積 → `small_logo_candidate`
  - BBox 面積 > 35% 圖面積 → `large_logo_candidate`
  - 偵測到 >1 個 Logo → `multi_logo_candidate`
- 未偵測到（detections 為空）：**soft_pass**，標記 `difficulty_flags: ["no_logo_prescreen_detection", "hard_logo_candidate"]`
- 模型不可用：**skipped**

Gate 輸出範例（soft_pass）：

```
{"gate": "logo_prescreen", "outcome": "soft_pass",
 "detections": [], "model_id": "/raid/ming/logo/engine/model_cache/yolov8s-worldv2.pt"}
```

本批次 10 筆通過記錄全部為 **soft_pass**，代表 YOLO-World 在這批 Marketplace 縮圖上未找到明確 Logo，但因為 soft_pass 機制，這些「困難案例」仍被保留，並以 `hard_logo_candidate` 標記，留待 SAM3 精細處理。

實際輸出（`meta/staging_gate.json`，精簡）：

```
{"seen_images": 40, "updated_images": 40,
 "quality_status_counts": {"filtered": 30, "passed": 10},
 "report": {
   "failure_reason_counts": {"near_duplicate": 30},
   "gate_stats": {
     "size_format":    {"passed": 40, "failed": 0,  "soft_pass": 0},
     "blur":           {"passed": 40, "failed": 0,  "soft_pass": 0},
     "dedupe":         {"passed": 10, "failed": 30, "soft_pass": 0},
     "clip_logo":      {"passed": 6,  "failed": 0,  "soft_pass": 4},
     "logo_prescreen": {"passed": 0,  "failed": 0,  "soft_pass": 10}
   }
 }
}
```

8 9 Export Passed & 最終平衡

Step 8（export-image-records）：從 staging DB 撈出所有 `quality_status='passed'` 的記錄，輸出為 `meta/passed_records.json`。本批次共 10 筆。

Step 9（balance-records，第二次）：對 10 筆通過記錄再次做品牌/類別平衡，輸出最終 `fetch/records.json`（10 筆）。

過關數量驗證

系統會檢查 `passed_count ≥ target_records`（10 ≥ 10），條件成立才繼續。 若不成立，Pipeline 立即報錯退出，提示使用者提高 `--oversample-factor`。

10 SAM3 Logo Segmentation — 精細像素級分割

目的：對最終 10 筆圖片，執行三層漸進式 Logo 定位與分割，產出精細的 pixel-level Logo Mask。

對應指令：

```
python logo_segmentation_pipeline.py \
    --input-records  fetch/records.json \
    --output         segment/records_with_logo_masks.json \
    --mask-dir       segment/masks/ \
    --viz-dir        segment/visualizations/ \
    --brand-records  brand/brand_records.json
```

Object-First 三層偵測架構

Layer 1：GroundingDINO 物件偵測（Object-First）

使用 `IDEA-Research/grounding-dino-tiny` 先偵測圖片中的主要物件（非 Logo）， 縮小後續 Logo 偵測的搜尋範圍。物件 Prompt 依據 category 與品牌動態組合：

category	object_terms（部分）	expected_object_fraction_range
apparel	shirt, tee, hoodie, jacket, clothing	[0.06, 0.98]
shoes	shoe, sneaker, footwear, boot	[0.06, 0.98]
mugs	mug, cup, tumbler, coffee cup	[0.05, 0.75]

Layer 2：GroundingDINO Logo 偵測（在物件 Crop 上）

對 Layer 1 找到的物件框，裁切出 Crop 後再用 Logo 相關 Prompt 偵測：

```
logo_prompt_variants = [
    "Adidas logo",           "Adidas brand logo",
    "Adidas emblem",         "Adidas mark",
    "Cloudfoam logo",        "Cloudfoam brand logo",
    "Adidas Cloudfoam logo","Adidas shirt logo",
    "Adidas logo on shirt", "shirt logo"
]
```

expected_logo_fraction_range：`[0.0001, 0.23]`（Logo 占圖比例合理範圍，太大或太小的偵測框會被排除）

實際偵測輸出範例（Adidas 上衣）：

```
{"logo_grounding_label": "shirt logo",
 "logo_grounding_score": 0.365249,
 "logo_grounding_bbox_xyxy": [71.24, 79.65, 183.4, 202.29],
 "logo_grounding_bbox_area": 13755.2,
 "logo_grounding_bbox_fraction": 0.12701,
 "logo_grounding_model_id": "IDEA-Research/grounding-dino-tiny"}
```

Layer 3：SAM3 精細 Mask 分割

以 Layer 2 的 Logo BBox 作為 Point Prompt，由 SAM3（`Sam3LogoSegmenter`）在**原始全圖**上產生 pixel-level binary mask：

```
{"logo_bbox_xyxy": [71.24, 79.65, 183.4, 202.29],
 "logo_bbox_area": 13755.2,
 "logo_mask_area_pixels": 10874,
 "logo_mask_area_fraction": 0.10041,
 "logo_segmentation_iou_prediction": 0.000751,
 "logo_segmentation_status": "ok",
 "logo_mask_path": "segment/masks/7d5ce5b96afddd7f.png",
 "logo_visualization_path": "segment/visualizations/7d5ce5b96afddd7f.png"}
```

輸出說明

輸出	格式	說明
`segment/records_with_logo_masks.json`	JSON 陣列	原始圖片記錄 + 所有 SAM3 偵測欄位
`segment/masks/<id>.png`	灰階 PNG	Binary Mask：白色=Logo 區域，黑色=背景
`segment/visualizations/<id>.png`	RGB PNG	原圖疊加 Mask 半透明遮罩 + BBox 框線的視覺化

實際輸出統計（`meta/segment.json`）：

```
{"records": 10, "ok": 10, "no_detection": 0,
 "missing_image": 0, "download_failed": 0,
 "model_unavailable": 0, "inference_error": 0}
```

SAM3 分割成功率

本批次 10/10（100%）成功分割，零失敗。每筆 Mask 平均覆蓋 Logo BBox 面積的約 79%（`mask_area_pixels=10874` vs `bbox_area=13755`），符合精細分割預期。

11 Phase1 Workflow — 核心標注引擎

目的：對最終 10 筆圖片執行完整的 AI 標注管線，產生 Logo Instances 並寫入正式 engine DB。

本批次使用 SAM3 Enrich 模式：因 `--with-sam3` 已在 Step 10 產生分割結果， Phase1 的 annotation 子步驟採用 `annotate_from_segment_records(enrich=True)`， 直接從 SAM3 結果讀取 BBox 與 Mask，再疊加 OCR/CLIP/VLM 進行豐富化。

11a. Seed Ontology（engine DB）

將 brand_records.json 中 13 個品牌寫入正式 engine DB，tier=`"silver"`。

```
{"inserted": 13, "tier": "silver", "dry_run": false}
```

11b. Ingest Images（engine DB）

10 筆圖片記錄寫入正式 engine DB。

```
{"seen": 10, "inserted": 10, "skipped_existing": 0,
 "skipped_duplicate": 0, "phash_failures": 0}
```

11c. Quality Gate（engine DB）

對正式 DB 中的圖片再次執行 Gate（`resume=True`，已 gated 的跳過）。本批次全 10 筆通過：

```
{"seen": 10, "status_counts": {"passed": 10}, "failure_reason_counts": {}}
```

11d. Annotate（SAM3 Enrich 模式）

對每筆 `logo_segmentation_status="ok"` 的 SAM3 記錄，依序執行以下步驟：

1. 裁切 Logo Crop

從 SAM3 輸出的 `logo_bbox_xyxy`（或 `logo_grounding_bbox_xyxy`）裁切出 Logo 區域， 作為後續 OCR 與 CLIP 的輸入。

```
x0, y0, x1, y1 = 71, 79, 183, 202   # Adidas 上衣範例
logo_crop = image.crop((71, 79, 183, 202))
```

2. PaddleOCR — Logo 文字辨識

對 Logo Crop 執行 OCR，嘗試辨識品牌文字（如 "ADIDAS"、"STARBUCKS"）。

PaddleOCR 輸入 / 輸出

- **輸入：**原始圖片路徑 + `crop_box_xyxy`（裁切範圍）
- **輸出：**`{"engine":"paddleocr","text":"ADIDAS","confidence":0.92,"lines":[...]}`
- 若 Logo 為純圖形（無文字）：`{"text":null,"confidence":0.0,"lines":[]}`

本批次多數記錄 `ocr_text=null`，原因是 Poshmark 縮圖中 Logo 文字過小或角度傾斜，PaddleOCR 未能辨識出有效文字。

3. CLIP Brand Retrieval — Logo 品牌比對

以 Logo Crop 圖片為 Query，對品牌知識庫中每個品牌的文字 Prompt 計算 CLIP 分數，取最高分作為品牌歸屬依據。

CLIP Retrieval 輸入 / 輸出

- **圖片端：**Logo Crop（PIL Image）
- **文字端（每個品牌）：**`"a photo of the {alias} logo"`（使用所有 aliases_en）
- **例如 Adidas：**`"a photo of the Cloudfoam Comfort shoes logo"`
- **輸出：**按分數排序的 top-N matches 清單

實際輸出範例（Disney Mug 圖的 Logo Crop，CLIP 比對 Adidas）：

```
{"matches": [
  {"brand_id": "adidas",       "score": 0.2501, "prompt": "a photo of the Cloudfoam Comfort shoes
  {"brand_id": "lululemon",    "score": 0.2495, "prompt": "a photo of the lululemon logo"},
  {"brand_id": "samsung",      "score": 0.2491, "prompt": "a photo of the Samsung brand logo"},
  {"brand_id": "under-armour", "score": 0.2469, "prompt": "Under Armour, Inc. logo"},
  {"brand_id": "nike",         "score": 0.2409, "prompt": "Nike, Inc. logo"}
], "model_id": "openai/clip-vit-base-patch32"}
```

CLIP 歧義注意

本批次中，CLIP 各品牌分數差異極小（差距 < 0.01），系統將此標記為 `ambiguity_note: "ambiguous_clip_brand_retrieval"`，並降低 confidence（最終 confidence ≈ 0.63），觸發 `review_status=needs_review`。

4. LLaVA VLM — 圖像自然語言描述（主要 Caption）

僅對 `quality_status=passed` 的記錄執行。使用 `llava-hf/llava-1.5-7b-hf`（7B 參數多模態模型）對整圖與 Logo Crop 各生成一段描述。

LLaVA 輸入 Prompt 設計

目標	Prompt 文字
整圖描述	`"Describe the brand, object, and photo scene."`
Logo Crop 描述	`"Describe the logo, brand, and object in this crop."`

實際輸出範例（Disney Mug 整圖）：

```
full_image_caption:
"USER: \nDescribe the brand, object, and photo scene.
ASSISTANT: The image features a Goofy Cuisine coffee mug, which is a cartoon-themed
mug with a picture of Goofy, a popular character from the Disney universe. The mug
is placed on a white surface, possibly a table or a bed. The scene is set in a room"
```

Logo Crop 描述範例（Adidas Logo Crop）：

```
logo_crop_caption:
"USER: \nDescribe the logo, brand, and object in this crop.
ASSISTANT: The image features a close-up of a black and white logo, which appears
to be a combination of a letter and a graphic. The logo is placed on a white
background, making it stand out. The brand name is not visible in the image,
but the logo itself is the main focus."
```

BLIP 備用機制

當 `--use-vlm` 啟用時，LLaVA 為主要 Caption 引擎；BLIP（`Salesforce/blip-image-captioning-base`）為備用。若 VLM 回傳錯誤，系統自動 fallback 至 BLIP 的輸出。兩者的 prompt 相同，但 BLIP 為非對話式模型，輸出格式無 `USER:/ASSISTANT:` 標記。

5. merge_signals() — 多信號品牌決策融合

整合 brand_hint（collector 標籤）、OCR 文字、CLIP 分數、VLM Caption，依照以下優先順序決定最終品牌歸屬：

1. **brand_hint + OCR 一致**：最高信心，confidence = 0.95+，attribution_source = `"ocr_match"`

2. **CLIP 第一名且清晰領先**（gap > 0.02）：高信心，attribution_source = `"clip_retrieval"`
3. **CLIP 歧義**（各品牌分數接近）：標記 `ambiguous_clip_brand_retrieval`，降低 confidence
4. **Brand Hint 作為保底**：若 CLIP/OCR 均不確定，採用 collector 的 brand_hint

最終融合結果範例（Adidas 圖，CLIP 歧義）：

```
{"attribution_source": "clip_retrieval",
 "merged_brand_id":   "adidas",
 "merged_brand_name": "Adidas",
 "confidence":        0.6251,
 "ambiguity_note":    "ambiguous_clip_brand_retrieval"}
```

11e. Review Queue 輸出

對所有 `confidence < 0.9` 的 Logo Instances，輸出待人工審核清單。

```
python -m logo_data_engine --db engine/logo_engine.db review-queue --output review/review_queue.j
```

本批次 20 筆 Logo Instances 全數 `needs_review`（confidence 範圍約 0.62–0.75）。

實際輸出（`meta/phase1.json`，精簡）：

```
{"ontology":   {"inserted": 13},
 "images":     {"seen": 10, "inserted": 10},
 "gate":       {"quality_status_counts": {"passed": 20}},
 "annotation": {"seen": 10, "inserted": 10},
 "review":     {"records": 20}}
```

12 Export Knowledge Base

目的：將 engine DB 中的 `image_records` 與 `logo_instances` 做 JOIN，輸出一份完整的、自描述的 JSON 知識庫，供下游訓練、分析或人工審核使用。

對應指令：

```
python -m logo_data_engine --db engine/logo_engine.db \
    export-kb --output export/knowledge_base.json
```

實際輸出統計：

```
{"output": "export/knowledge_base.json", "records": 20}
```

> 20 筆 = 10 張圖 × 每張平均 2 個 Logo Instances（部分圖片偵測到多個 Logo）。

13 Analyze Run — 批次分析報告

> **目的：**對整個批次執行結果做結構化分析，產生 Markdown 報告與 JSON 摘要。
>
> **對應指令：**
>
> ```
> python -m logo_data_engine --db engine/logo_engine.db \
>  analyze-run \
>  --run-dir  results/batch_10_vlm3_sam3 \
>  --output-md  analysis/analysis.md \
>  --output-json analysis/summary.json \
>  --target-records 10
> ```
>
> 報告內容包含：品牌/類別/來源分佈表、品質狀態統計、Logo Instance Tier 分佈、目標達成確認。 詳見第 6 節。

4 輸出目錄結構

```
results/batch_10_vlm3_sam3/
│
├── engine/
│   └── logo_engine.db              ← 正式 SQLite DB（image_records + logo_instances + brand_records）
│
├── staging/
│   └── logo_engine.db              ← 暫存 DB（Quality Gate 前的候選管理）
│
├── fetch/
│   ├── records.json                ← 最終 10 筆記錄（品質過關 + 平衡後）
│   └── images/                     ← 所有下載的圖片（候選池 + 歷史）
│       ├── adidas_69c47adb.jpg
│       ├── disney_69c9e3a9.jpeg
│       └── ...（共 60+ 張）
│
├── brand/
│   └── brand_records.json          ← 品牌知識庫（Wikidata 資料，13 個品牌）
│
├── segment/                        ← SAM3 分割輸出（--with-sam3 時產生）
│   ├── records_with_logo_masks.json ← 含 logo_bbox, mask_path, SAM3 統計的完整記錄
│   ├── masks/                      ← Binary Logo Masks（PNG 灰階）
│   │   ├── 7d5ce5b96afddd7f.png    ← 以 record_id 命名
│   │   └── ...（共 20 張）
│   └── visualizations/             ← 視覺化疊圖（原圖 + Mask + BBox）
│       ├── 7d5ce5b96afddd7f.png
│       └── ...（共 20 張）
│
├── review/
│   └── review_queue.json           ← 待人工審核的 Logo Instances（20 筆，全 needs_review）
│
├── export/
│   └── knowledge_base.json         ← 最終交付 JSON（20 筆，含完整 metadata + logo instances）
│
├── analysis/
│   ├── analysis.md                 ← Markdown 批次分析報告
│   ├── summary.json                ← 結構化統計摘要（JSON）
│   └── analyze.json                ← analyze-run 原始輸出 log
│
└── meta/                           ← 每個步驟的執行 log（JSON）
    ├── preflight.json              ← 模型可用性預檢結果
    ├── batch_plan.json             ← limit_per_pair 計算結果
    ├── raw_records.json            ← Collector 原始記錄（34 筆）
    ├── collector.json              ← collector 執行統計
    ├── balance.json                ← 第一次平衡統計（30 筆候選）
    ├── candidate_records.json      ← 30 筆候選記錄
    ├── ontology.json               ← Ontology 抓取 log
    ├── staging_seed.json           ← Staging seed-ontology log
    ├── staging_ingest.json         ← Staging ingest-images log
```

```
    ├── staging_gate.json          ← Quality Gate 詳細結果（40 筆）
    ├── export_passed.json         ← export-image-records log
    ├── passed_records.json        ← 10 筆通過 gate 的記錄
    ├── balance_passed.json        ← 第二次平衡 log
    ├── segment.json               ← SAM3 分割統計
    ├── phase1.json                ← Phase1 各子步驟統計
    ├── export.json                ← export-kb log
    └── summary.json               ← DB 整體狀態摘要
```

5 Knowledge Base 欄位完整說明

`export/knowledge_base.json` 是本 Pipeline 的最終交付物。每筆記錄由 `image_records` 與 `logo_instances` JOIN 而成，包含以下欄位：

5.1 圖片層欄位（來自 image_records）

欄位名稱	類型	範例值	說明
`image_id`	string	`"20cf1fe51128a440"`	系統內部圖片 ID（8 bytes hex）
`brand`	string	`"Disney"`	Collector 標記的品牌（brand_hint）
`category`	string	`"mugs"`	商品類別
`source`	string	`"poshmark"`	資料來源名稱
`source_channel`	string	`"marketplace"`	管道類型
`source_kind`	string	`"marketplace"`	影響 Quality Gate soft-pass 決策
`product_name`	string	`"Walt Disney Mug Goofy Cuisine..."`	商品標題
`product_url`	string	`"https://poshmark.com/listing/..."`	商品頁面 URL
`image_url`	string	`"https://di2ponv0v5otw.cloudfront.net/..."`	原始圖片 CDN URL
`local_image_path`	string	`"/raid/ming/logo/.../disney_69c9e3a9.jpeg"`	本地圖片完整路徑
`crawl_timestamp_utc`	ISO 8601	`"2026-04-04T01:13:44.996800+00:00"`	爬取時間戳（UTC）
`engine_tier`	string	`"proposal"`	資料品質等級：proposal / silver / gold
`engine_quality_status`	string	`"passed"`	Quality Gate 最終結論
`engine_quality_score`	float	`0.9016`	各關卡分數加權平均（0–1）
`engine_quality_gate`	object	見下方	各關卡詳細結果（含分數）
	array		

欄位名稱	類型	範例值	說明
`difficulty_flags`		`["hard_logo_candidate", ...]`	特殊難例標記（影響訓練難度分層）
`knowledge_slots`	object	`{"brand_aliases":[], ...}`	品牌知識槽（待後續填充）

5.2 Logo Instance 層欄位（來自 logo_instances，巢狀於 `logo_instances[]`）

欄位名稱	類型	範例值	說明
`instance_id`	string	`"e3a406bcd1ae34c2"`	Logo Instance 唯一 ID
`brand_id`	string	`"adidas"`	canonical 品牌 ID（slug 格式）
`merged_brand_name`	string	`"Adidas"`	merge_signals() 決定的品牌名稱
`detector_name`	string	`"imported_grounding_sam3"`	偵測器名稱（SAM3 模式）
`bbox`	array[4]	`[141.61, 64.95, 169.13, 75.24]`	Logo BBox（xyxy 格式，像素座標）
`mask_path`	string	`"segment/masks/27b53c8d432fc606.png"`	SAM3 Binary Mask 路徑
`detector_score`	float	`0.365249`	GroundingDINO Logo 偵測信心分數
`ocr_text`	string/null	`null`	PaddleOCR 辨識文字（null=未辨識到）
`ocr_confidence`	float	`0.0`	OCR 信心分數
`clip_score`	float	`0.2501`	CLIP Retrieval 最高匹配分數
`caption_text`	string	LLaVA 輸出文字	Logo Crop 的 VLM 描述（優先）
`caption_model`	string	`"llava-hf/llava-1.5-7b-hf"`	使用的 Caption 模型
`confidence`	float	`0.6251`	merge_signals() 的最終信心分數
`ambiguity_note`	string/null	`"ambiguous_clip_brand_retrieval"`	歧義說明（觸發 needs_review）
`review_status`	string	`"needs_review"`	人工審核狀態

欄位名稱	類型	範例值	說明
`tier`	string	`"proposal"`	資料等級
`attribution`	object	見說明	完整 attribution chain（CLIP/VLM/merged）
`provenance`	object	見說明	偵測來源追溯（grounding model、prompt 等）

6 本批次統計結果

6.1 品牌與類別分佈

品牌	類別	筆數
Adidas	apparel	2
Disney	mugs	2
Adidas	shoes	1
Lululemon	apparel	1
New Balance	shoes	1
Starbucks	mugs	1
Under Armour	apparel	1
Under Armour	shoes	1
合計		**10**

6.2 品牌分佈（fetch/records.json 中的 10 筆）

品牌	筆數	佔比
Adidas	3	30%
Disney	2	20%
Under Armour	2	20%
Lululemon	1	10%
New Balance	1	10%
Starbucks	1	10%
合計	**10**	**100%**

6.3 類別分佈

類別	筆數	佔比
apparel	4	40%
shoes	3	30%
mugs	3	30%
合計	**10**	**100%**

6.4 Knowledge Base 完整統計

指標	數值
Knowledge Base 記錄數（image × instance JOIN）	20
唯一品牌數	6
唯一類別數	3
唯一資料來源數	1（poshmark）
quality_status = passed	20（100%）
quality_status = filtered	0
Logo Instance Tier = proposal	20（100%）
review_status = needs_review	20（100%）
review_status = auto_accept	0
SAM3 Mask 數量	20
OCR 辨識到文字	0（縮圖過小）
目標達成（target_achieved）	True

7 Review Queue 與下一步行動

7.1 Review Queue 說明

`review/review_queue.json` 包含所有 `review_status != "auto_accept"` 的 Logo Instances，依 confidence 由低至高排序，優先讓人工審核最不確定的記錄。

每筆 review queue 記錄包含：

- `instance_id`、`image_id`：用於定位記錄
- `brand`：merge_signals() 的品牌決策
- `detector_score`：GroundingDINO 信心
- `clip_score`：CLIP Retrieval 最高分
- `confidence`：最終綜合信心（本批次約 0.62–0.75）
- `local_image_path`：本地圖片路徑，供人工查看
- `image_record`：完整的原始記錄 metadata

低信心範例記錄：

```
{"instance_id": "ce4d772ce18196d0", "brand": "Nike", "confidence": 0.6191,
 "detector_score": 0.219449, "clip_score": 0.23817,
 "ambiguity_note": "ambiguous_clip_brand_retrieval",
 "review_status": "needs_review",
 "local_image_path": "fetch/images/adidas_69bff709.jpeg"}
```

品牌標籤不一致案例

上例中，圖片本身是 **Adidas**（brand_hint），但 CLIP Retrieval 最高分判定為 **Nike**。這是 CLIP 歧義的典型案例，需人工確認正確品牌標籤。

7.2 人工審核流程

審核後，對每筆記錄填入 `review_status`（`accepted` 或 `rejected`）及 `tier`（`gold` 或 `silver`），儲存為新的 JSON 後執行：

```
python -m logo_data_engine --db engine/logo_engine.db \
    review-apply --decisions review/reviewed_decisions.json
```

7.3 建議後續動作

優先順序	動作	說明
1（立即）	人工審核 Review Queue	20 筆全 needs_review，尤其關注 brand_hint 與 CLIP 決策不一致的記錄
2	查看 segment/visualizations/	確認 SAM3 Mask 覆蓋範圍是否合理，標記明顯分割錯誤
3	增加 --oversample-factor	目前 30 筆候選中有 30 筆 near_duplicate，可嘗試 --oversample-factor 5 擴大來源多樣性
4	擴增品牌/類別 Pairs	目前 17 pairs，可加入更多品牌或新類別（如 bags、electronics）
5	使用 import-run 匯入	此批次審核完成後，可用 `import-run` 合併至全局 engine DB

本報告由 Logo Data Engine 自動產生並人工整理。

批次目錄：`/raid/ming/logo/logo_data_engine/results/batch_10_vlm3_sam3`

文件生成時間：2026-04-04